

Guía de Aprendizaje: Exclusión Mutua (Mutex)

Malak Sanchez

Monitorías de Sistemas Operativos

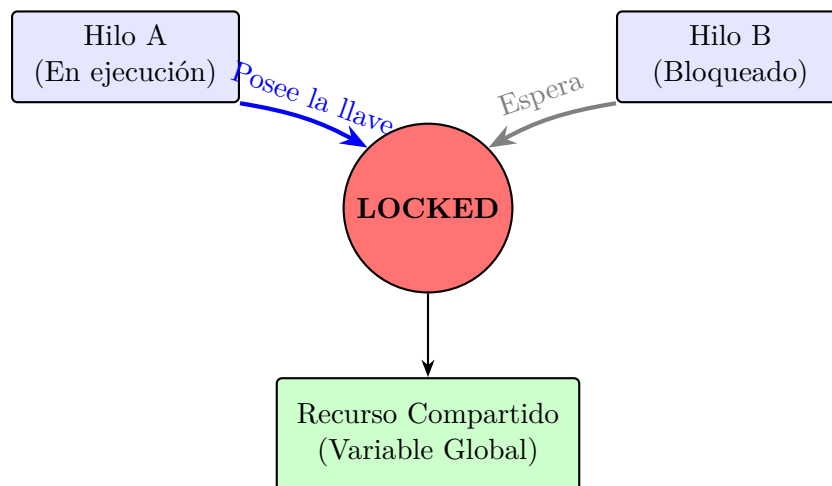
13 de mayo de 2026

1. El Problema: Condición de Carrera

Cuando múltiples hilos acceden y modifican un recurso compartido (como una variable global) al mismo tiempo, el resultado final puede depender del orden impredecible de ejecución. Esto se conoce como **Condición de Carrera** (Race Condition).

2. La Solución: Mutex

Un **Mutex** (MUTual EXclusion) es un mecanismo de sincronización que asegura que solo un hilo a la vez pueda acceder a una sección crítica de código. Es como una llave única: solo el hilo que tiene la llave puede entrar a la habitación; los demás deben esperar afuera.



3. Funciones Principales

```
#include <pthread.h>

pthread_mutex_t mutex;

// Inicializacion
pthread_mutex_init(&mutex, NULL);

// Bloquear (Entrar a la seccion critica)
pthread_mutex_lock(&mutex);

// Desbloquear (Salir de la seccion critica)
pthread_mutex_unlock(&mutex);

// Destruir
pthread_mutex_destroy(&mutex);
```

4. Ejemplo: Contador Seguro

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  int contador = 0;
5  pthread_mutex_t lock;
6
7  void* incrementar(void* arg) {
8      for(int i = 0; i < 1000000; i++) {
9          pthread_mutex_lock(&lock);
10         contador++;
11         pthread_mutex_unlock(&lock);
12     }
13     return NULL;
14 }
15
16 int main() {
17     pthread_t t1, t2;
18     pthread_mutex_init(&lock, NULL);
19
20     pthread_create(&t1, NULL, incrementar, NULL);
21     pthread_create(&t2, NULL, incrementar, NULL);
22
23     pthread_join(t1, NULL);
24     pthread_join(t2, NULL);
25
26     printf("Valor final del contador: %d\n", contador);
27     pthread_mutex_destroy(&lock);
28     return 0;
29 }
```

5. Ejercicios

1. ¿Qué sucede si olvida llamar a `pthread_mutex_unlock`? Pruebe el comportamiento.
2. Implemente un programa donde varios hilos retiren dinero de una cuenta bancaria compartida, asegurando que el saldo nunca sea negativo.